

EventZone (Foundation) - Capstone Project for Certification

Contents

Purpose

- 1) About Us
- 2) Problem Statement
- 3) Roles
- 4) In-Scope Features
- 5) User Flows (Happy Path)
- 7) Architecture
- 8) Tech Stack
- 9) Data Model
- 10) API Outline
- 11) NFRs & SLAs
- 12) Testing
- 13) Engineering PoV
- 14) Delivery Plan
- 15) Sample Acceptance Criteria
- 16) Example Seed Data (SQL)
- 17) Simple Wireframe
- 18) Sample User Stories
- 19) Definition of Ready (DoR) & Done (DoD)
- 20) Evaluation Rubric (Aligned to Foundation Skills)
- 22) Stretch Goals (Optional, If Time Permits)
- 23) Getting Started (Suggested Commands)

Purpose

Build an event ticket booking web application where users can browse upcoming events (concerts, sports, workshops), choose a ticket category, and register/book tickets. Event organisers manage their event listings. Admins manage categories. The focus is on foundational full-stack skills - React/Angular, Spring Boot, REST, PostgreSQL, Git, and basic testing - aligned with the Foundation skill matrix.

1) About Us

At EventZone, we connect people with the events they love. This Foundation version delivers a clean event-browsing and ticket-booking experience - no payment gateway, no QR code generation, just a complete registration and booking flow.

2) Problem Statement

Event-goers need a simple way to discover upcoming events, see ticket categories with pricing, and register for tickets without contacting organisers manually.

Goal: Deliver a minimal yet complete event booking flow demonstrating full-stack fundamentals.

3) Roles

1. Attendee/User - Can browse events, view details, book tickets, and view My Bookings.
2. Organiser - Can create and manage their own events and ticket categories.
3. Admin - Can manage event categories and deactivate events or users.

Note: Role stored as String field: String role; // "ATTENDEE" | "ORGANISER" | "ADMIN"

Role Flag / Field Approach

- User entity: String role; (defaults to ATTENDEE on registration)
- ORGANISER accounts are seeded or promoted by ADMIN.
- Backend: check role in controller before allowing event/category CRUD.
- Frontend: Organiser dashboard visible only to ORGANISER; Admin panel to ADMIN.

Backend Concept

- Backend: check role in controller before allowing event/category CRUD.
- Frontend: Organiser dashboard visible only to ORGANISER; Admin panel to ADMIN.

Frontend Concept

- Backend: check role in controller before allowing event/category CRUD.
 - Frontend: Organiser dashboard visible only to ORGANISER; Admin panel to ADMIN.
-

4) In-Scope Features

Authentication (Basic)

- Register, Login, Logout.
- Simple password hashing (BCrypt).

Event Catalog

- List events with title, category, date, venue, cover image, and ticket price range.
- Filter by category (Concert / Sports / Workshop / Conference).
- Event detail page with full description, ticket categories, and seats available.

Ticket Booking

- Select a ticket category (e.g. General / VIP) and quantity (1-5).
- Confirm booking -> Booking ID generated.
- Booked quantity deducted from available capacity.
- Cancel a booking (capacity restored).

Organiser Dashboard

- Create/edit/delete events (title, description, date, venue, coverImageUrl, categoryId).
- Add ticket categories per event (name, price, totalSeats).
- View total bookings per ticket category.

Admin - Category Management

- CRUD for event categories.
- Deactivate/activate events.

5) User Flows (Happy Path)

1. Browse: Attendee opens site -> sees event list -> filters by category.
2. Detail: Clicks event -> sees description + ticket categories with prices.
3. Auth: If not logged in -> prompted to register/login.
4. Book: Selects ticket category + quantity -> confirms -> sees Booking ID.
5. My Bookings: Views all bookings -> can cancel a booking.
6. Organiser: Logs in -> creates event -> adds ticket categories.
7. Admin: Logs in -> manages categories -> deactivates an event.

7) Architecture

- Frontend: React (Hooks + Context API) or Angular (Services + Components). Routing enabled.
- Backend: Spring Boot (REST Controllers, Services, Spring Data JPA).
- Database: PostgreSQL (Dev may use H2 for quick start).
- Auth: Stateless JWT or simple session cookie (choose one; document trade-offs).
- State: Selected ticket category and quantity in frontend state.
- Deployment: Localhost

8) Tech Stack

Frontend

- React + Vite/CRA (Routing, Hooks, Context) or Angular (Routing, Services)
- HTML5, CSS3, JavaScript (ES6), TypeScript

Backend

- Java 17+, Spring Boot, Spring Web, Spring Data JPA, Validation
- JUnit5, Mockito

Database

- PostgreSQL, JPA/Hibernate
- H2 for local

Tools & Practices

- Git Basics (branching, PRs), Postman/Swagger, Agile ceremonies
- AI Pair Programming: GitHub Copilot/ChatGPT for boilerplate and explanations

9) Data Model

Entities & Relationships

- EventCategory (1) -- (M) Event
- Event (1) -- (M) TicketCategory
- User (1) -- (M) Booking
- TicketCategory (1) -- (M) Booking
- Booking stores: ticketCategoryId, quantity, status, bookingRef

Note: TicketCategory tracks availableSeats. On booking: availableSeats -= quantity. On cancel: availableSeats += quantity.

10) API Outline

Auth

- POST /api/auth/register - {email, password, name} -> 201
- POST /api/auth/login -> 200 {token}
- POST /api/auth/logout -> 204

Events

- GET /api/events - list with optional ?category=
- GET /api/events/{id} - detail + ticket categories
- POST /api/events (organiser) - create
- PUT /api/events/{id} (organiser/admin)
- DELETE /api/events/{id} (organiser/admin)

Ticket Categories

- POST /api/events/{id}/ticket-categories (organiser) - {name, price, totalSeats}
- PUT /api/ticket-categories/{id} (organiser)
- DELETE /api/ticket-categories/{id} (organiser)

Bookings

- POST /api/bookings - {ticketCategoryId, quantity} -> creates booking
- GET /api/bookings/mine - attendee's bookings
- PUT /api/bookings/{id}/cancel - cancel booking

Error Format (Consistent)

```
{
  "timestamp": "2025-09-12T12:00:00Z",
  "path": "/api/...",
  "error": "VALIDATION_ERROR",
  "message": "Field is required"
}
```

11) NFRs & SLAs

- Performance:
 - API response ≤ 2 s under light load (1-5 concurrent users locally).
 - UI renders initial content within 3 s on localhost.
- Security:
 - Passwords hashed (BCrypt); JWT/session token stored securely.
 - Basic input validation & server-side checks.
- Scalability/Concurrency:
 - Test with 5-10 concurrent requests locally. Simulate using Postman Runner or multiple browser tabs. No JMeter or load-testing tools required at this level.
- Error Handling:
 - User-friendly error toasts/messages on UI; structured error JSON on API.
- Logging:
 - Console logs; simple request/response logs (no external logging infra).
- Documentation:
 - README.md with setup steps, seed data, and run commands.
 - API contracts documented with Swagger/OpenAPI.

12) Testing

- Backend Unit Tests
 - Services & Repos with JUnit5 + Mockito
 - Target coverage: $\geq 60\%$ (method-level)
- API Testing
 - Manual via Postman or Swagger.
- Frontend
 - Optional unit tests for 1-2 key components (e.g., Booking form).
- No automated regression/e2e required.

13) Engineering PoV

- Code Quality:
 - Follows SOLID basics where practical (e.g., service layer, DTOs).
 - No critical lint errors (ESLint/TSLint).
- Branching:
 - main (stable), feature/* branches; PR reviews by peers/mentor.
- Coverage:
 - Aim 60-70% backend, frontend minimal.

- Dependencies:
 - Keep list short; beware of transitive vulnerabilities.
 - Docs:
 - API endpoints documented; Postman collection included.
-

14) Delivery Plan (3 Sprints, 1-2 weeks each)

Sprint 1 - Foundations & Event Catalog

- Scaffolding (FE + BE), DB setup, Git workflow
- EventCategory, Event entities; GET /events list and detail
- Frontend: Home + Event List + Event Detail
- Seed data loading

Sprint 2 - Auth & Ticket Booking

- Register/Login; protect booking and organiser routes
- Ticket category display; booking flow; My Bookings page; cancel booking
- Capacity decrement/restore logic; validation + error handling

Sprint 3 - Organiser Dashboard & Admin

- Organiser: create/edit events + add ticket categories + view booking counts
 - Admin: category CRUD + event deactivation
 - Polish: README, Postman collection, unit tests >=60%
-

15) Sample Acceptance Criteria

Event Listing

- Given I am a guest, I see at least 5 seeded events with title, category, date, and image.
- Filter by category returns only matching events.

Ticket Booking

- Given I book 2 General tickets, availableSeats for that category decreases by 2.
- Cancelling restores the 2 seats to available.

Organiser Dashboard

- Organiser can create an event and it appears in the catalog immediately.
- Organiser sees total bookings per ticket category.

Admin

- Only admin can access /admin.
 - Admin can create a category and it is available when an organiser creates an event.
-

16) Example Seed Data (SQL)

SQL

```
INSERT INTO users(id,email,password_hash,role,name,created_at)
VALUES (gen_random_uuid(),'admin@eventzone.com','$2a$10$hash','ADMIN','Admin',now()),
```

```
(gen_random_uuid(), 'org1@eventzone.com', '$2a$10$hash', 'ORGANISER', 'Arjun Events', now()),
(gen_random_uuid(), 'user1@eventzone.com', '$2a$10$hash', 'ATTENDEE', 'Divya', now());

INSERT INTO event_category(id,name) VALUES
(gen_random_uuid(), 'Concert'), (gen_random_uuid(), 'Sports'), (gen_random_uuid(), 'Workshop');

INSERT INTO event(id,title,description,event_date,venue,cover_image_url,organiser_id,category_id,created_at)
VALUES
(gen_random_uuid(), 'Rock Night 2025', 'Live rock concert by top bands',
'2025-11-15 19:00:00', 'HICC Hyderabad', 'https://picsum.photos/seed/rock/400',
(SELECT id FROM users WHERE email='org1@eventzone.com'),
(SELECT id FROM event_category WHERE name='Concert'), now());

INSERT INTO ticket_category(id,event_id,name,price,total_seats,available_seats,created_at)
VALUES
(gen_random_uuid(), (SELECT id FROM event WHERE title='Rock Night 2025'), 'General', 999.00, 200, 200, now()),
(gen_random_uuid(), (SELECT id FROM event WHERE title='Rock Night 2025'), 'VIP', 2499.00, 50, 50, now());
```

17) Simple Wireframe

- Home/List: Category filter tabs + Grid of event cards [Cover] Title - Date - Venue - Price from X
- Event Detail: [Cover] Title - Description - Date/Venue - Ticket Categories (Name/Price/Available) - [Book]
- Booking Modal: Select Category + Quantity dropdown + Total + [Confirm Booking]
- My Bookings: Table (Event / Category / Qty / Booking Ref / Status) - [Cancel]
- Organiser Dashboard: My Events table + Add Event form + Add Ticket Category form

18) Sample User Stories

- As an attendee, I want to browse events by category so I can find events I like.
- As an attendee, I want to see ticket categories and prices so I can book within my budget.
- As an attendee, I want to confirm a booking so I can secure my spot at the event.
- As an organiser, I want to create an event and set ticket categories so attendees can book.
- As an admin, I want to manage categories so the event catalog stays organised.

19) Definition of Ready (DoR) & Done (DoD)

DoR

- User story has clear acceptance criteria.
- API/DB changes identified (if any).
- Mock/UX reference exists (even a sketch).

DoD

- Code compiles; unit tests for backend services pass.
- Manual tests completed (Postman + UI).
- No critical ESLint/TS issues; meaningful README update.
- Peer review completed; merged to main.

20) Evaluation Rubric (Aligned to Foundation Skills)

Area	What We Check	Weight
Frontend (React/Angular)	Routing, hooks/services, component communication, state handling (Context/Service), basic TypeScript usage	25%
Backend (Spring Boot)	Clean REST controllers, services, JPA entities & relationships, validation, basic error handling	25%
Common Skills	Core Java correctness, SQL/JPA usage, SOLID basics, Git hygiene, HTML/CSS semantics	20%
Testing	JUnit/Mockito for services/repos; Postman collection; target $\geq 60\%$ coverage	15%
Generative AI Usage	Appropriate use of Copilot/ChatGPT for boilerplate, explanations, refactoring (cite prompts in README)	10%
Documentation & Readme	Setup steps, seed data, API list, constraints & trade-offs	5%

22) Stretch Goals (Optional, If Time Permits)

- Countdown timer to event date on the event detail page.
- Booking confirmation page with a shareable booking reference.
- Organiser revenue summary (total bookings x price per category).

23) Getting Started (Suggested Commands)

Backend

- Java 17+, Spring Boot.
- Profiles: dev (H2), prod (Postgres).

Frontend

- React: `npm create vite@latest -ui -- --template react-ts`
- Angular: `ng new -ui --routing --style=scss`

Run Order

1. Start DB (H2 or Postgres)
 2. Run Spring Boot
 3. Seed Data
 4. Run UI
-